Smart with

Nathaniel Burke

TU Delft, Netherlands, n.burke@tudelft.nl

Motivation The with construct, proposed by McBride and McKinna [22], extends dependent pattern matching [15] with a mechanism for matching on intermediary computations, whilst generalising the context to support dependent elimination. This feature is implemented in Agda under the name with-abstractions. While useful, the feature can also be a footgun. Proving simple laws about definitions by with-abstraction is often fiddly and sometimes impossible due to failures in the generalisation step producing *ill-typed with-abstraction* errors [1].

To illustrate the problem, and our solution, consider the following attempt to prove that filtering a list twice with the same predicate returns the same result as filtering once.[1]

```
filter : (A → Bool) → List A → List A
filter f []          = []
filter f (x ,- xs) with f x
... | tt = x ,- filter f xs
... | ff = filter f xs
```

```
filter-twice : filter f (filter f xs) ≡ filter f xs
filter-twice            {xs = []}       = refl
filter-twice {f = f} {xs = x ,- xs} with f x
... | tt = {!!}
... | ff = filter-twice {xs = xs}
```

The base case of filter-twice is easy: the match on {xs = []} substitutes xs for [] in the context, allowing the filter applications on both sides of the goal to reduce to [].

The inductive case is harder. Because filter is defined by with-abstraction, filter-twice must repeat the match on f x to make progress. In the false case, the goal reduces to exactly the induction hypothesis, but in the true case (left as a hole, {!!}), Agda displays the goal type as filter f (x ,- filter f xs) | f x ≡ x ,- filter f xs. The "| f x" syntax on the left-hand side denotes that the outer filter application is still stuck on f x, rather than reducing to x ,- filter f (filter f xs) as we might hope.

The problem is due a fundamental limitation in how with-abstractions support dependent elimination. Generalisation (during which the matched term is abstracted over) is a one-off transformation so, **critically**, Agda does not "remember" the equality between f x and tt. Indeed, if we with-abstract over f x once more, Agda will ask us to handle both the case when f x is tt and when it is ff, but we already know it is tt!

Since version 2.6.2, Agda builds in the *inspect idiom*, allowing us to write "in eq" after a with-abstraction to bind a propositional identity between the abstracted term and the pattern. In the present example, this feature enables us to complete the proof by with-abstracting over f x once more, simultaneously with eq to refute the false case:

```
filter-twice {f = f} {xs = x ,- xs} with f x in eq
... | ff = filter-twice {xs = xs}
... | tt with f x | eq
... | tt | eq = ap (x ,-_) (filter-twice {xs = xs})
... | ff | () -- absurd
```

Even so, the workaround is tedious, and sometimes does not work at all.

[1] The examples in this abstract are also available self-contained [8].

Inspired by the smart case proposal of Altenkirch et al. [2], we propose an improved with-abstraction mechanism for Agda. On our work-in-progress branch [7], Agda accepts directly filling the above hole with ap (x ,-_) (filter-twice {xs = xs}).

Our implementation builds upon (a subset[2] of) local rewrite rules as proposed by Leray and Winterhalter [20] and recently implemented in Agda [11]. To elaborate with-abstractions, Agda's existing generalisation procedure first normalises the context (and goal), then applies a one-off rewrite (replacing occurrences of the matched term with a fresh argument) and finally checks the context is still well-typed. Smart with, on the other hand, simply extends the context with a local rewrite rule from the matched term to the fresh argument (and this rewrite rule is refined in the clauses). By no longer having to normalise the entire context, we are also hopeful that smart with can be made more performant than Agda's existing with construct.

Indexed datatypes and rewrite Smart with is especially powerful when working with indexed datatypes. Consider the following definition of natural numbers indexed by parity:

```
data Parity : Set where
  even : Parity
  odd  : Parity

inv   : Parity → Parity
_xor_ : Parity → Parity
      → Parity

data Nat : Parity → Set where
  ze : Nat even
  su : Nat (inv p) → Nat p
```

We will try to implement addition of Nats, _+_ (which xors the respective parities) and prove that ze is a right-identity of addition, +ze. The base case of _+_ is easy, but the inductive step requires us to do some work. inv p xor q is stuck, so we must prove a lemma about how inv commutes with xor (concretely, inv-xor : inv p xor q ≡ inv (p xor q)).

This is reasonable enough, but we then face a more technical challenge in how to actually use this lemma. Aiming for convenience, we gravitate towards Agda's rewrite construct. We first with-abstract over the recursive call in order to get some variable into the context (n+m) with type Nat (inv p xor q). Then, we rewrite its type to the required Nat (inv (p xor q)) and apply successor.

```
_+_ : Nat p → Nat q → Nat (p xor q)
_+_ {p} {q} (su n) m
  with    n+m ← n + m
  rewrite inv-xor {p} {q}
  = su n+m
```

```
+ze : n + ze ≡[ ap Nat xor-even ]≡ n
+ze {n = su {p} n}
  with    n′ ← n + ze -- in eq
  rewrite inv-xor {p} {even}
  = {!!}
```

Note that inlining n+m does not work. rewrite desugars to simultaneous with-abstractions (over the left-hand-side and identity proof) and therefore inherits the same limitations: Agda does not "remember" the equality between inv p xor q and inv (p xor q).

Proving +ze is trickier. Technically, the theorem is not even type correct without knowing xor-even : p xor even ≡ p but proving this is easy enough. We then state the type of +ze with a dependent identity type. In the inductive case, we first need to repeat the same with-abstraction and rewrite to make su n + ze reduce. The remaining goal ({!!}) asks us to prove su n' ≡[ap Nat xor-even]≡ su n, but this is impossible: we have lost the connection between n' and n + ze. If we try to apply the inspect idiom again and bind "in eq" in +ze, we hit the dreaded *ill-typed with-abstraction* error: inv p xor even != lhs of type Parity when checking that the type ... of the generated with function is well-formed.

Again, the problem is with-abstractions' one-off nature. For eq : n' ≡ n + ze to typecheck, both n' and n + ze must have the same type. This is the case until we apply the rewrite. n''s type is rewritten, but n + ze is left alone, and now the context no longer typechecks.

[2] We only need *ground* rules: all variables in equations are bound in the context or via higher-order matching.

Finally, we give a taste of a more practical application. Many useful models of type theory are generated by interpreting types as instances of some algebraic structure. For example, the groupoid [18] and presheaf [3, 4] models of type theory interpret types as groupoids and presheaves respectively. Actually, to account for type dependency, we interpret contexts as groupoids/presheaves and types as (fibrant) displayed groupoids/displayed presheaves.

When trying to mechanise these constructions in a proof assistant, this dependency causes a "transport-hell" problem: the groupoid/functor laws only hold propositionally, and so we must insert transports in the interpretation of types.[3] When defining the interpretation of terms, these transports need to be manually shifted around, adding significant clutter to the proofs [10]. With smart with/smart rewrite, we can instead reflect these equations in each case and have the transports reduce away, getting us much closer to the clarity of informal presentations.

Theory We propose a target type theory for a language featuring smart with by way of introducing a new contextual judgement for convertibility assumptions, following [2]. We denote a context Γ extended by an equation between terms t_1, t_2 with $\Gamma \rhd \mathsf{t}_1 \sim \mathsf{t}_2$.

Similar judgemental-equality-extending constructs have been explored as an alternative to judgemental computation rules [24], in the setting of dependent Haskell [26, 21] and more generally (by the name cofibrations) in cubical type theory [14, 5] and synthetic ∞-category theory [23], as well as to control unfolding [17] and to implement record patching [27].

We bind convertibility assumptions with a form of *local equality reflection*: given a propositional identity $\mathsf{p} : \mathsf{t}_1 = \mathsf{t}_2$ we bind both $\mathsf{t}_1 \sim \mathsf{t}_2$ and $\mathsf{p} \sim \mathsf{refl}$ in the body. To retain good metatheoretical properties (subject reduction, decidable typechecking etc.) we make reflection *second-class*, only available at the top-level. This is in keeping with Agda's generative pattern matching and gives us the freedom to syntactically restrict equations (e.g. disallowing overlap), without endangering stability under substitution.

The theory for Boolean equations was explored in earlier work [9]. Since then, we have worked to clarify and extend the normalisation argument. There are two main components: building rewrite environments and normalisation by evaluation (NbE). We build rewrite environments from the convertibility assumptions in the telescope of every top-level definition. This step is inherently very syntactic (smart with checking is not stable under conversion). NbE modulo a particular rewrite environment rewrites β-neutral terms to values during unquote/reflect (conditional on neutral/normal form comparisons). Staying *algebraic* (working with type theory as a quotient inductive-inductive type) during NbE appears feasible, but requires care to avoid circularity; we cannot depend on injectivity of type formers during normalisation.

Equations at more complicated types than Booleans pose some further difficulties. For example, the natural number equation $\mathsf{n} \sim \mathsf{su}\ (\mathsf{f}\ \mathsf{n})$ must be oriented towards $\mathsf{su}\ (\mathsf{f}\ \mathsf{n})$ to unblock β-reductions, but loops if we apply it naively. We are also investigating whether stabilised neutrals [25] can help to further abstract the proof (moving from presheaves on thinnings to presheaves on renamings).

Finally, we touch on conservativity/consistency. Local equality reflection is implied by judgemental equality reflection, so a translation into ETT is immediate, but we would ideally have a translation into ITT that does not rely on function extensionality (we only allow reflecting individual equations, not families of equations) or UIP (for consistency with HoTT).

Naive smart with certainly does imply UIP. The situation somewhat resembles that of dependent pattern matching [15, 16, 6], where it is possible to avoid UIP by restricting the unification algorithm [12], so we are hopeful a similar criterion on reflected equations could be employed in our setting (perhaps disallowing reflexive equations).

[3] Type theory's own substitution calculus can also be a source of "transport-hell", but *strictification* via global rewrite rules [13] or Kaposi and Pujet's construction [19] can mostly resolve problems of this kind.

References

[1] The Agda Team. *With-Abstraction. The Agda 2.8.0 User Manual*. 2024. URL: https://agda.readthedocs.io/en/v2.8.0/language/with-abstraction.html (visited on 05/01/2026).

[2] Thorsten Altenkirch. "The case of the smart case. How to implement conditional convertibility?" In: *Presented at NII Shonan Seminar 007*. 2011. URL: https://shonan.nii.ac.jp/archives/seminar/007/files/2011/09/altenkirch_slides.pdf.

[3] Thorsten Altenkirch, Martin Hofmann, and Thomas Streicher. "Categorical Reconstruction of a Reduction Free Normalization Proof". In: *Category Theory and Computer Science, 6th International Conference, CTCS '95, Cambridge, UK, August 7-11, 1995, Proceedings*. Ed. by David H. Pitt, David E. Rydeheard, and Peter T. Johnstone. Vol. 953. Lecture Notes in Computer Science. Springer, 1995, pp. 182–199. DOI: 10.1007/3-540-60164-3_27.

[4] Thorsten Altenkirch and Ambrus Kaposi. "Normalisation by Evaluation for Type Theory, in Type Theory". In: *Log. Methods Comput. Sci.* 13.4 (2017). DOI: 10.23638/LMCS-13(4:1)2017.

[5] Carlo Angiuli et al. "Syntax and models of Cartesian cubical type theory". In: *Math. Struct. Comput. Sci.* 31.4 (2021), pp. 424–468. DOI: 10.1017/S0960129521000347.

[6] Bruno Barras et al. "A New Elimination Rule for the Calculus of Inductive Constructions". In: *Types for Proofs and Programs, International Conference, TYPES 2008, Torino, Italy, March 26-29, 2008, Revised Selected Papers*. Ed. by Stefano Berardi, Ferruccio Damiani, and Ugo de'Liguoro. Lecture Notes in Computer Science. Springer, 2008, pp. 32–48. DOI: 10.1007/978-3-642-02444-3_3.

[7] Nathaniel Burke. *Agda/local-rewrite-matches. Branch on Agda GitHub Fork*. 2026. URL: https://github.com/NathanielB123/agda/tree/local-rewrite-matches (visited on 03/31/2026).

[8] Nathaniel Burke. *Examples.agda*. 2026. URL: https://github.com/NathanielB123/TYPES-2026/blob/main/Examples.agda (visited on 04/01/2026).

[9] Nathaniel Burke. "Local Rewriting in Dependent Type Theory". Master's thesis. Imperial College London, 2025. URL: https://github.com/NathanielB123/fyp/blob/main/submitted.pdf.

[10] Nathaniel Burke. *Some experiments with mechanised metatheory of type theory. GitHub Repository*. See specifically `Models/GrpdModel.agda` and `NonLinNbE/Eval.agda` for examples of the "transport-hell" problem. 2026. URL: https://github.com/NathanielB123/TT (visited on 03/30/2026).

[11] Nathaniel Burke and Jesper Cockx. *#8385: Local Rewrite Rules. Agda GitHub PR*. 2026. URL: https://github.com/agda/agda/pull/8385 (visited on 03/26/2026).

[12] Jesper Cockx, Dominique Devriese, and Frank Piessens. "Eliminating dependent pattern matching without K". In: *J. Funct. Program.* 26 (2016), e16. DOI: 10.1017/S0956796816000174.

[13] Jesper Cockx, Nicolas Tabareau, and Théo Winterhalter. "The taming of the rew: a type theory with computational assumptions". In: *Proc. ACM Program. Lang.* 5.POPL (2021), pp. 1–29. DOI: 10.1145/3434341.

[14] Cyril Cohen et al. "Cubical Type Theory: A Constructive Interpretation of the Univalence Axiom". In: *21st International Conference on Types for Proofs and Programs, TYPES 2015, Tallinn, Estonia, May 18-21, 2015*. Ed. by Tarmo Uustalu. Vol. 69. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2015, 5:1–5:34. DOI: 10.4230/LIPICS.TYPES.2015.5.

[15] Thierry Coquand. "Pattern matching with dependent types". In: *Informal proceedings of Logical Frameworks*. Vol. 92. 1992, pp. 66–79. URL: https://wonks.github.io/type-theory-reading-group/papers/proc92-coquand.pdf.

[16] Healfdene Goguen, Conor McBride, and James McKinna. "Eliminating Dependent Pattern Matching". In: *Algebra, Meaning, and Computation, Essays Dedicated to Joseph A. Goguen on the Occasion of His 65th Birthday*. Ed. by Kokichi Futatsugi, Jean-Pierre Jouannaud, and José Meseguer. Vol. 4060. Lecture Notes in Computer Science. Springer, 2006, pp. 521–540. DOI: 10.1007/11780274_27.

[17] Daniel Gratzer et al. "Controlling unfolding in type theory". In: *Mathematical Structures in Computer Science* 35 (2025), e38. DOI: 10.1017/S0960129525100327.

[18] Martin Hofmann and Thomas Streicher. "The groupoid interpretation of type theory". In: *Twenty-five years of constructive type theory (Venice, 1995)*. Oxford Logic Guides 36 (1998), pp. 83–111.

[19] Ambrus Kaposi and Loïc Pujet. "Type Theory in Type Theory using a Strictified Syntax". In: *Proc. ACM Program. Lang.* 9.ICFP (2025), pp. 855–885. DOI: 10.1145/3747535.

[20] Yann Leray and Théo Winterhalter. "Encode the Cake and Eat it Too". preprint. 2025. URL: https://hal.science/hal-05160846.

[21] Yiyun Liu and Stephanie Weirich. "Dependently-Typed Programming with Logical Equality Reflection". In: *Proc. ACM Program. Lang.* 7.ICFP (2023), pp. 649–685. DOI: 10.1145/3607852.

[22] Conor McBride and James McKinna. "The view from the left". In: *J. Funct. Program.* 14.1 (2004), pp. 69–111. DOI: 10.1017/S0956796803004829.

[23] Emily Riehl and Michael Shulman. "A type theory for synthetic ∞-categories". In: *Higher Structures* 1 (1 2017), pp. 147–224. DOI: 10.21136/hs.2017.06.

[24] Vilhelm Sjöberg and Stephanie Weirich. "Programming up to Congruence". In: *Proceedings of the 42nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2015, Mumbai, India, January 15-17, 2015*. Ed. by Sriram K. Rajamani and David Walker. ACM, 2015, pp. 369–382. DOI: 10.1145/2676726.2676974.

[25] Jonathan Sterling and Carlo Angiuli. "Normalization for Cubical Type Theory". In: *36th Annual ACM/IEEE Symposium on Logic in Computer Science, LICS 2021, Rome, Italy, June 29 - July 2, 2021*. IEEE, 2021, pp. 1–15. DOI: 10.1109/LICS52264.2021.9470719.

[26] Stephanie Weirich et al. "A specification for dependent types in Haskell". In: *Proc. ACM Program. Lang.* 1.ICFP (2017), 31:1–31:29. DOI: 10.1145/3110275.

[27] Tesla Zhang. "Three non-cubical applications of extension types". In: *CoRR* abs/2311.05658 (2023). DOI: 10.48550/ARXIV.2311.05658.